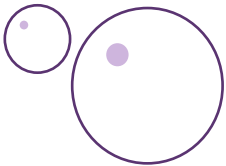
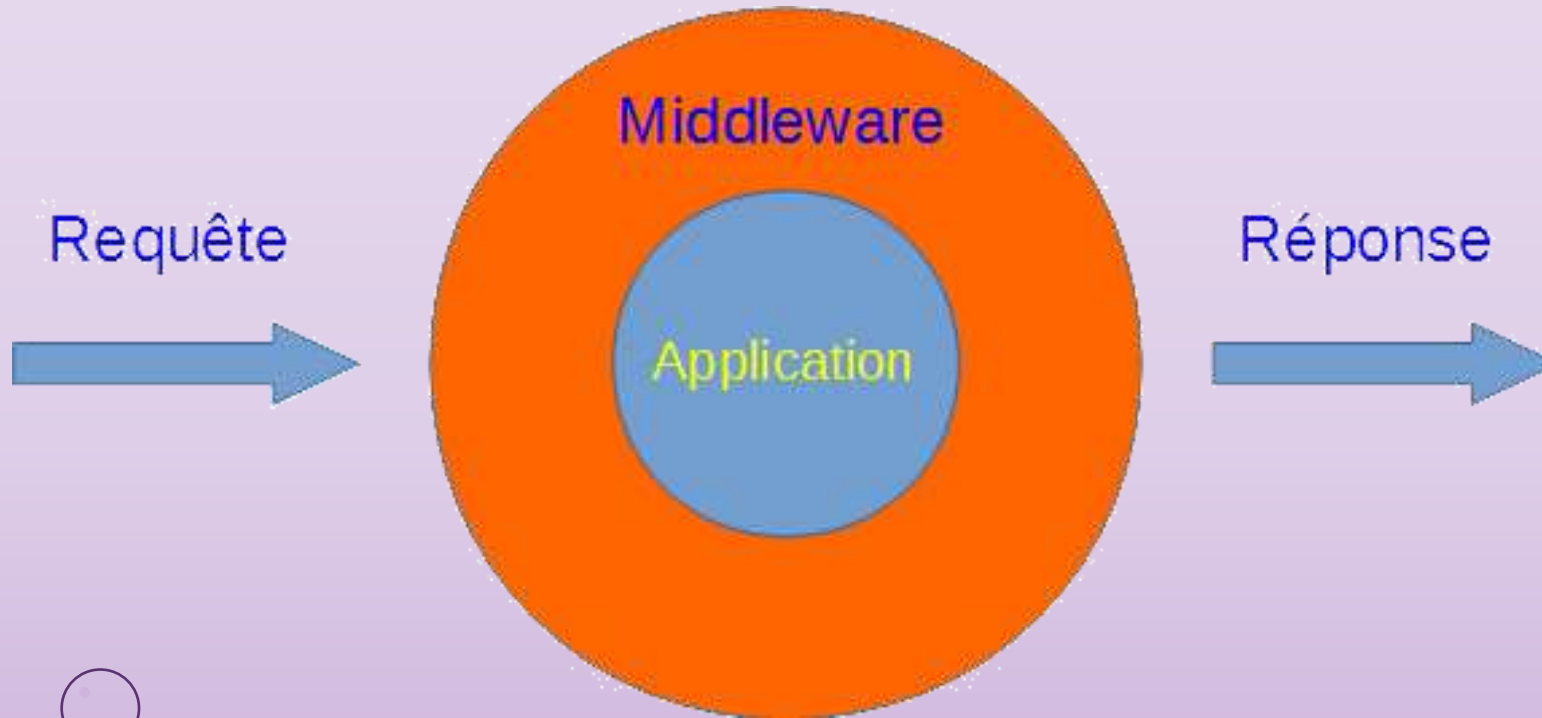


Utilisation des Middleware



Définition

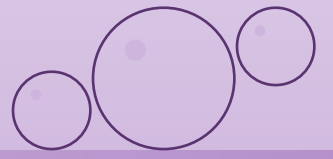
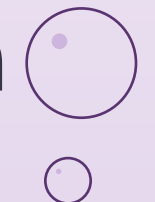
Le middleware fournit un mécanisme pratique pour inspecter et filtrer les requêtes HTTP entrant dans votre application.





Définition

Par exemple, Laravel inclut un middleware qui vérifie que l'utilisateur de votre application est **authentifié**. Si l'utilisateur n'est pas authentifié, le middleware redirigera l'utilisateur vers l'écran de connexion de votre application. Cependant, si l'utilisateur est authentifié, le middleware permettra à la demande de continuer plus loin dans l'application.



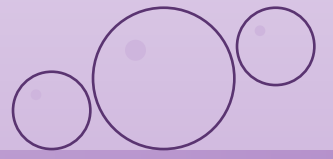
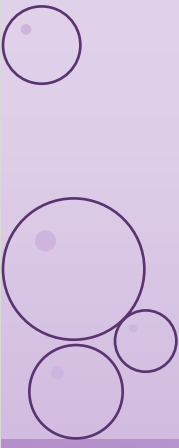


Créer un nouveau middleware

Pour créer un nouveau middleware, utilisez la commande Artisan make:middleware



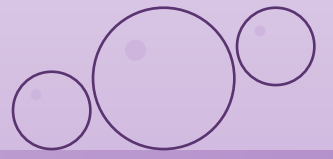
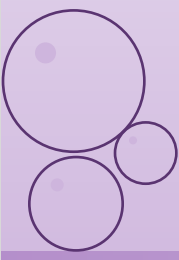
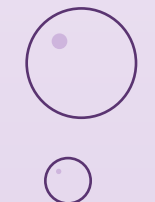
```
php artisan make:middleware EnsureTokenIsValid
```





Créer un nouveau middleware

Cette commande placera une nouvelle classe **EnsureTokensValid** dans votre répertoire **app/Http/Middleware**. Dans ce middleware, nous n'autoriserons l'accès à la route que si l'entrée token fournie correspond à une valeur spécifiée. Sinon, nous redirigerons les utilisateurs vers l'URI home.



```
<?php

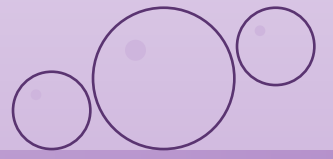
namespace App\Http\Middleware;
use Closure;
use Illuminate\Http\Request;

class EnsureTokenIsValid
{
    /**
     * ...
     */
    public function handle(Request $request, Closure $next)
    {
        if ($request->input('token') !== 'my-secret-token'){
            return redirect('home');
        }
        return $next($request);
    }
}
```



Créer un nouveau middleware

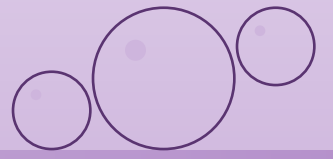
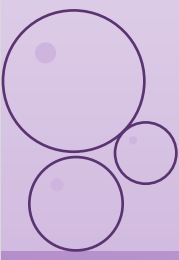
Comme vous pouvez le voir, si le donné token ne correspond pas à notre jeton secret, le middleware renverra une redirection HTTP au client ; sinon, la demande sera transmise plus loin dans l'application. Pour transmettre la demande plus profondément dans l'application (permettant au middleware de "passer"), vous devez appeler le rappel **\$next** avec le **\$request**.

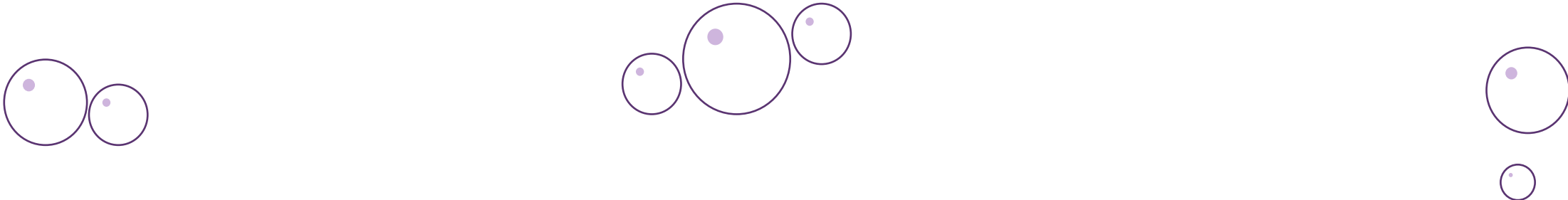




Créer un nouveau middleware

Il est préférable d'envisager le middleware comme une série de "couches" que les requêtes HTTP doivent traverser avant d'atteindre votre application. Chaque couche peut examiner la demande et même la rejeter entièrement.





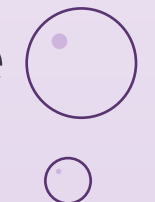
Enregistrement



Middleware Globale

V9

Si vous souhaitez qu'un middleware s'exécute lors de chaque requête HTTP adressée à votre application, répertoriez la classe middleware dans la propriété **\$middleware** de votre classe **app/Http/Kernel.php**.



```
protected $middleware = [  
    // \App\Http\Middleware\TrustHosts::class,  
    \App\Http\Middleware\TrustProxies::class,  
    \Illuminate\Http\Middleware\HandleCors::class,  
    \App\Http\Middleware\EnsureTokenIsValid::class,  
    \App\Http\Middleware\PreventRequestsDuringMaintenance::class,  
    \Illuminate\Foundation\Http\Middleware\ValidatePostSize::class,  
];
```

Middleware Globale

V12

Dans Laravel 12 (et 11), les groupes de middlewares ne sont plus configurés dans **app/Http/Kernel.php** mais directement dans la méthode **withMiddleware** du fichier **bootstrap/app.php**.

```
->withMiddleware(function (Middleware $middleware): void {  
    $middleware->web(append: [  
        \App\Http\Middleware\EnsureTokenIsValid::class,  
    ]);  
})
```

Affectation de middleware aux routes

V9

Si vous souhaitez affecter un middleware à des routes spécifiques, vous devez d'abord affecter au middleware une clé dans le fichier **app/Http/Kernel.php** de votre application. Par défaut, la propriété **\$routeMiddleware** de cette classe contient des entrées pour le middleware inclus avec Laravel.

Affectation de middleware aux routes

V9

Vous pouvez ajouter votre propre middleware à cette liste et lui attribuer une clé de votre choix :

```
protected $routeMiddleware = [  
    'auth' => \App\Http\Middleware\Authenticate::class,  
    'token_test' => \App\Http\Middleware\EnsureTokenIsValid::class,  
    'auth.basic' => \Illuminate\Auth\Middleware\AuthenticateWithBasicAuth::class,  
    'auth.session' => \Illuminate\Session\Middleware\AuthenticateSession::class,  
    'cache.headers' => \Illuminate\Http\Middleware\SetCacheHeaders::class,  
    'can' => \Illuminate\Auth\Middleware\Authorize::class,  
    'guest' => \App\Http\Middleware\RedirectIfAuthenticated::class,  
];
```

Affectation de middleware aux routes

Une fois le middleware défini dans le noyau HTTP, vous pouvez utiliser la méthode `middleware` pour affecter le middleware à une route :

```
Route::get('/profile', function () {  
    //  
})->middleware('auth');
```

Affectation de middleware aux routes

Vous pouvez affecter plusieurs middlewares à la route en transmettant un tableau de noms de middlewares à la méthode `middleware` :

```
Route::get('/', function () {  
    //  
})->middleware(['first', 'second']);
```

Affectation de middleware aux routes

Lors de l'attribution du middleware, vous pouvez également transmettre le nom complet de la classe :

```
use App\Http\Middleware\EnsureTokenIsValid;  
  
Route::get('/profile', function () {  
    //  
})->middleware(EnsureTokenIsValid::class);
```


Affectation de middleware aux routes

V12

Vous utilisez cette méthode lorsque vous voulez un nom court (un alias) pour le middleware, ce qui est souvent plus propre, ou si vous devez appliquer le même middleware à partir d'un paramètre.

```
$middleware->alias([  
    'checkToken' => EnsureTokenIsValid::class,  
]);
```

L'exclusion de middleware

Lors de l'attribution d'un middleware à un groupe de routes, vous devrez peut-être parfois empêcher le middleware de s'appliquer sur une route individuelle au sein du groupe. Vous pouvez accomplir cela en utilisant la méthode **withoutMiddleware** :

```
Route::get('/profile', function () {  
    //  
    })->withoutMiddleware([EnsureTokenIsValid::class]);
```

L'exclusion de middleware

Vous pouvez également exclure un ensemble donné de middleware d'un groupe entier de définitions de route :

```
Route::withoutMiddleware([EnsureTokenIsValid::class])->  
group(function () {  
    Route::get('/profile', function () {  
        //  
    });  
});
```



L'exclusion de middleware

La méthode **withoutMiddleware** ne peut supprimer que le middleware de route et ne s'applique pas au **middleware global**.



L'exclusion de middleware global

V12

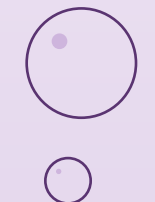
Pour exclure un middleware global, il faut utiliser la méthode **remove** dans la méthode **withMiddleware** du fichier **bootstrap/app.php**.

```
$middleware->remove([  
    VerifyCsrfToken::class  
]);
```

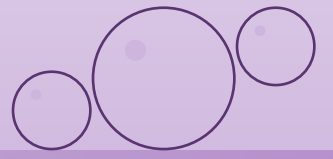
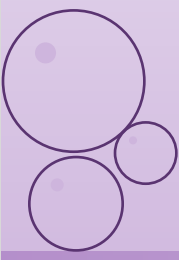


Groupes de middleware

Parfois, vous souhaitez peut-être regrouper plusieurs middlewares sous une seule clé pour faciliter leur affectation aux routes.



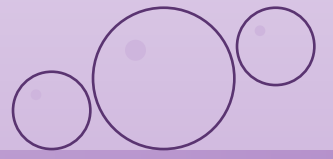
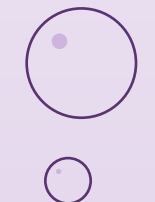
Vous pouvez accomplir cela en utilisant la propriété **\$middlewareGroups** de votre noyau HTTP.





Groupes de middleware

Prêt à l'emploi, **Laravel** est livré avec des groupes middlewares **Web** et **API** qui contiennent des middlewares communs que vous voudrez peut-être appliquer à vos routes Web et API. N'oubliez pas que ces groupes de middleware sont automatiquement appliqués par le fournisseur de services **App\Providers\RouteServiceProvider** de votre application aux routes dans vos fichiers de route **Web** et **API** correspondants.



```
protected $middlewareGroups = [
    'web' => [
        \App\Http\Middleware\EncryptCookies::class,
        \Illuminate\Cookie\Middleware\AddQueuedCookiesToResponse::class
    ],

    \Illuminate\Session\Middleware\StartSession::class,
    \Illuminate\View\Middleware\ShareErrorsFromSession::class,
    \App\Http\Middleware\VerifyCsrfToken::class,
    \Illuminate\Routing\Middleware\SubstituteBindings::class,

    'api' => [
        //
        \Laravel\Sanctum\Http\Middleware\EnsureFrontendRequestsAreStateful::class,
        'throttle:api',
        \Illuminate\Routing\Middleware\SubstituteBindings::class,
    ],
];
```




Groupes de middleware

Les groupes middlewares peuvent être affectés aux routes et aux actions du contrôleur en utilisant la même syntaxe que les middlewares individuels. Encore une fois, les groupes de middlewares facilitent l'affectation simultanée de plusieurs middlewares à une route :



```
Route::middleware(['groupe1'])->group(function () { /.../ });
```

```
Route::get('/', function () { //  
    })->middleware('groupe1');
```

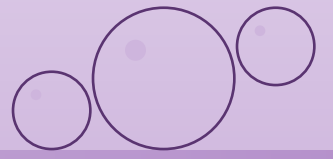


Le tri de middleware

Rarement, vous pouvez avoir besoin que votre middleware s'exécute dans un ordre spécifique, mais sans contrôler leur ordre lorsqu'ils sont affectés à la route.



Dans ce cas, vous pouvez spécifier votre priorité middleware en utilisant la propriété **\$middlewarePriority** de votre fichier **app/Http/Kernel.php**.



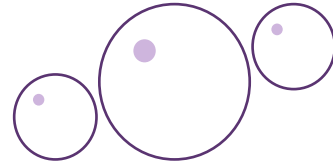
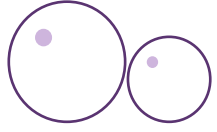


Le tri de middleware

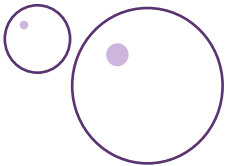
Cette propriété peut ne pas exister dans votre noyau HTTP par défaut. S'il n'existe pas, vous pouvez copier sa définition par défaut



```
/**
 * The priority-sorted list of middleware.
 *
 * This forces non-global middleware to always be in the given order.
 *
 * @var string[]
 */
protected $middlewarePriority = [
    \Illuminate\Foundation\Http\Middleware\HandlePrecognitiveRequests::class,
    \Illuminate\Cookie\Middleware\EncryptCookies::class,
    \Illuminate\Session\Middleware\StartSession::class,
    \Illuminate\View\Middleware\ShareErrorsFromSession::class,
    \Illuminate\Contracts\Auth\Middleware\AuthenticatesRequests::class,
    \Illuminate\Routing\Middleware\ThrottleRequests::class,
    \Illuminate\Routing\Middleware\ThrottleRequestsWithRedis::class,
    \Illuminate\Contracts\Session\Middleware\AuthenticatesSessions::class,
    \Illuminate\Routing\Middleware\SubstituteBindings::class,
    \Illuminate\Auth\Middleware\Authorize::class,
];
```



Paramétrage



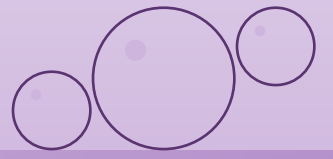


Paramétrage

Le middleware peut également recevoir des paramètres supplémentaires.

Par exemple, si votre application doit vérifier que l'utilisateur authentifié a un "**rôle**" donné avant d'effectuer une action donnée, vous pouvez créer un middleware

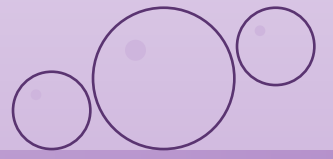
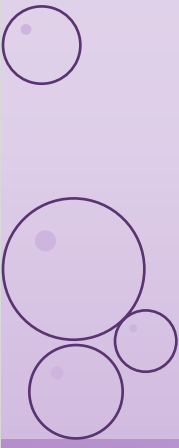
EnsureUserHasRole qui reçoit un nom de rôle comme argument supplémentaire.





Paramétrage

Des paramètres middleware supplémentaires
seront transmis au middleware après
l'argument **\$next**



```
namespace App\Http\Middleware;
use Closure;

class EnsureUserHasRole
{
    /**
     * Handle the incoming request.
     *
     * @param \Illuminate\Http\Request $request
     * @param Closure $next
     * @param string $role
     * @return mixed
     */
    public function handle($request, Closure $next, $role)
    {
        if (! $request->user()->hasRole($role)) { /* Redirect... */ }
        return $next($request);
    }
}
```


Paramétrage

Les paramètres du middleware peuvent être spécifiés lors de la définition de la route en séparant le nom du middleware et les paramètres par un : . Plusieurs paramètres doivent être délimités par des virgules :

```
Route::put('/post/{id}', function ($id) {  
    //  
    })->middleware('role:editor');
```